

Benefits of Using Apache Spark

Scalability: Spark can handle massive datasets by leveraging the computational power of multiple machines (or nodes) in a cluster.
Speed: In-memory processing capabilities allow Spark to perform data processing tasks much faster than traditional disk-based systems.

Fault Tolerance: Spark's resilient distributed datasets (RDDs) ensure data processing continues seamlessly even in the event of node failures.

Distributed Architecture of Spark:

Spark's architecture consists of a **master node (driver)** that coordinates the execution of tasks across multiple worker nodes (executors) in the cluster. Tasks are distributed across the cluster, with each executor responsible for processing a portion of the data independently.

This distributed architecture allows Spark to leverage the collective resources of the entire cluster, enabling parallel execution of computations and reducing overall processing time.

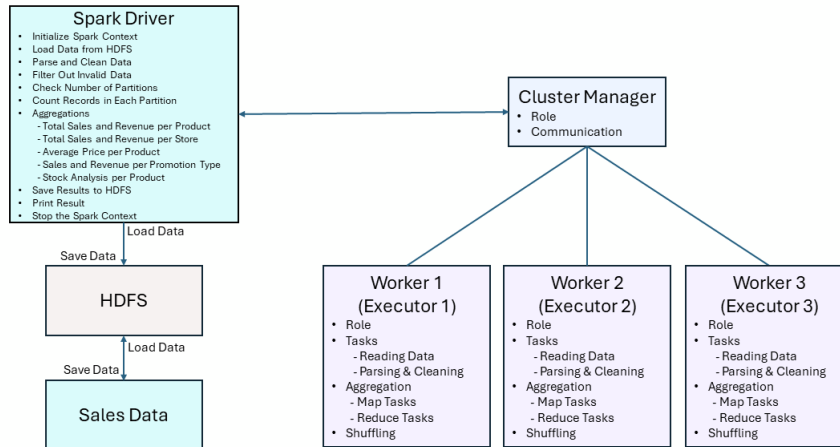


Figure: Distributed Architecture of Spark

Core Components:

Let us relate each component of Apache Spark's architecture to RetailWorld's example of processing nightly batch sales data:

Driver Program:

The driver program in RetailWorld's case is responsible for coordinating the Spark application's execution. It communicates with the cluster manager to request resources and coordinate the execution of tasks.

The driver program defines Resilient Distributed Datasets (RDDs) to represent the sales data stored in HDFS and specifies transformations and actions to clean, transform, and aggregate the data. Additionally, it creates the SparkContext to establish a connection with the cluster manager and coordinate the execution of Spark jobs for processing the sales data.

It handles the main logic of the application, including reading data, applying transformations, and collecting results.

Cluster Manager (e.g., Apache YARN):

In RetailWorld's scenario, the cluster manager (e.g., Apache YARN) is responsible for allocating resources across the cluster to process the nightly batch of sales data efficiently. It ensures that the necessary resources, such as CPU cores and memory, are available for the Spark application to execute its tasks effectively.

The Cluster Manager communicates with the Spark Driver to receive job details and assigns tasks to the worker nodes.

Task:

Tasks in RetailWorld's example represent the individual units of work that make up the nightly batch processing job. Each task operates on a partition of the sales data and is executed in parallel across the worker nodes in the cluster.

Tasks include data cleansing, transformation, aggregation, and analysis steps applied to the sales data RDD.

Worker Nodes:

Worker nodes in RetailWorld's scenario are the machines in the cluster responsible for executing tasks assigned by the driver program. Each worker node runs its own executor process. Executors on worker nodes execute tasks in parallel, processing data partitions assigned to them. They are responsible for running tasks and storing data needed by Spark applications.

Worker nodes execute tasks as assigned by the Cluster Manager. Each worker node runs an executor which processes partitions of the RDD.

The Tasks performed by the workernodes with respect to Retail Sales use case are:

Reading Data: Each worker node reads a partition of the input data from HDFS.

Parsing and Cleaning: Worker nodes parse and filter the data in parallel.

Aggregations:

- **Map Tasks:** Creating key-value pairs (e.g., (product_id, (sales, revenue))).
- **Reduce Tasks:** Aggregating data based on keys (e.g., summing sales and revenue per product).

Shuffling: During operations like reduceByKey, data is shuffled across nodes to group by key.

Workflow:

Initialization: The driver program initializes the SparkContext and establishes communication with the cluster manager.

Job Submission: The driver submits Spark jobs to the cluster manager, specifying RDDs, transformations, and actions to be executed.

Resource Allocation: The cluster manager allocates resources to the driver and executors based on the application's requirements, ensuring sufficient computing resources are available for processing the sales data efficiently.

Task Execution: Executors on worker nodes execute tasks in parallel, processing data partitions assigned to them. Each task performs specific data processing operations, such as cleaning outliers or aggregating sales.

Result Aggregation: Intermediate results from tasks are aggregated, and final results are returned to the driver program. Aggregated results, such as total sales per product or region, are collected and consolidated to generate actionable insights and reports for retail decision-making.

Cleanup: Once the job is completed, resources are released, and the SparkContext is shut down, concluding the nightly batch processing of sales data.

Author(s)

Lakshmi Holla
Malika Singla

